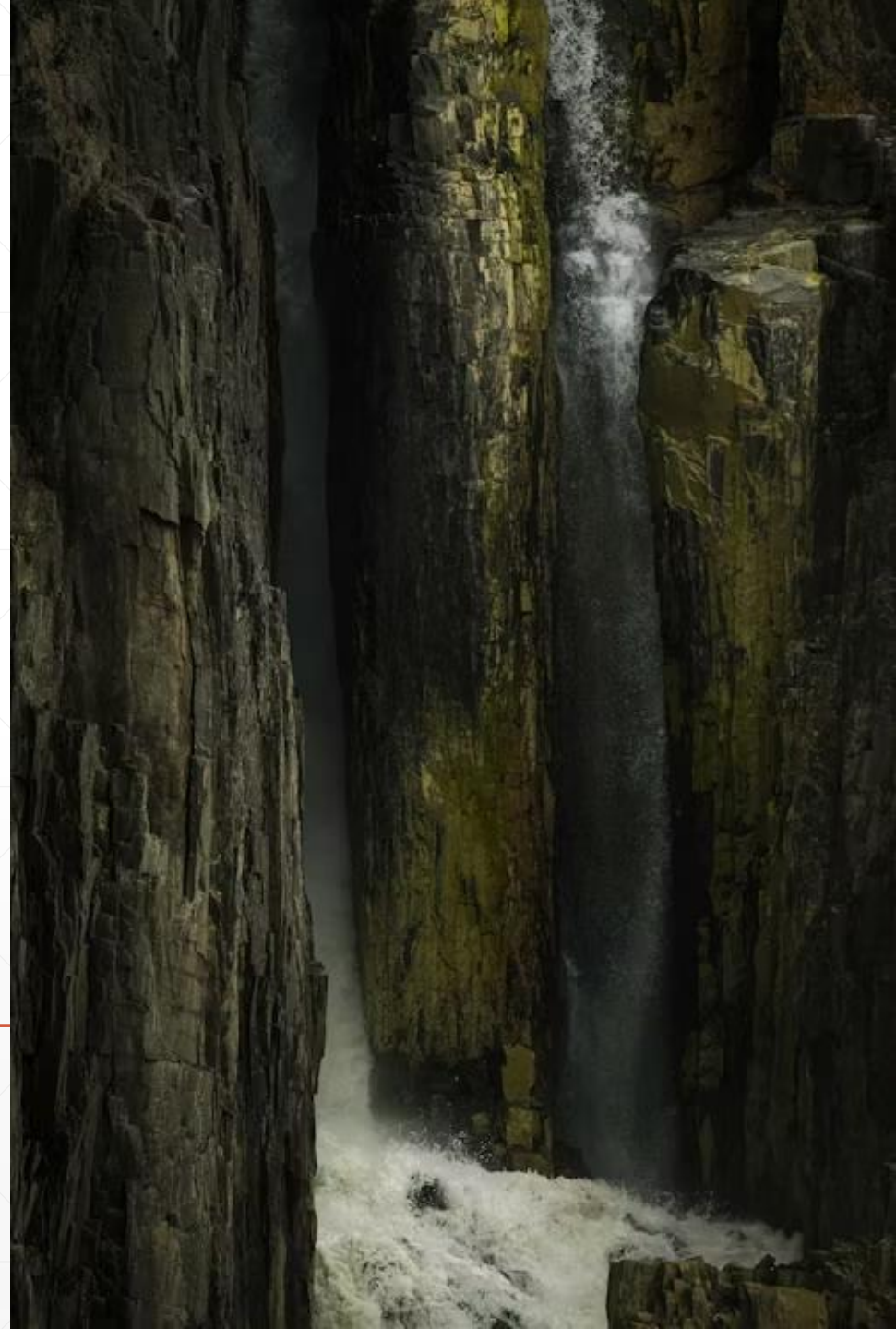


ByteBuf之三

ByteBuf数据存取

北京理工大学计算机学院
金旭亮



概述



前一讲介绍了如何实例化一个ByteBuf对象，本讲介绍如何向ByteBuf中存入和读取数据。



本讲所介绍的存入和读取数据的方法，是基础中的基础，在实际开发中被广泛使用，请务必掌握。

数据的读与写索引



ByteBuffer 维护了两个不同的索引 (readerIndex/writerIndex)：一个用于读取，一个用于写入。



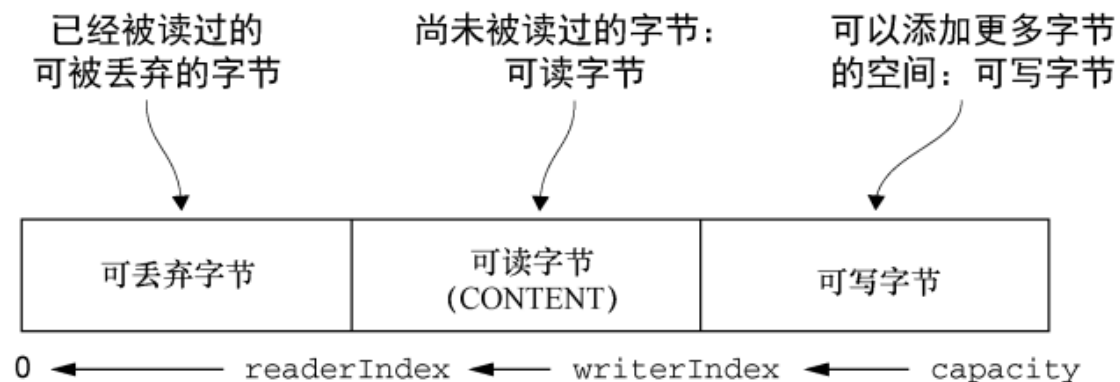
相应地，ByteBuffer可以读也可以写，读写的位置由readerIndex和writerIndex确定，随着读写操作，这两个索引值会不断地变化。



注意，readerIndex应该小于等于writerIndex，因为不能读取一个还没写入的数据，总有： $0 \leq \text{readerIndex} \leq \text{writerIndex} \leq \text{capacity}$ 。

ByteBuf中的“数据分段”

基于readerIndex、writerIndex和capacity，将ByteBuf自然地分为“三段”，如下所示：

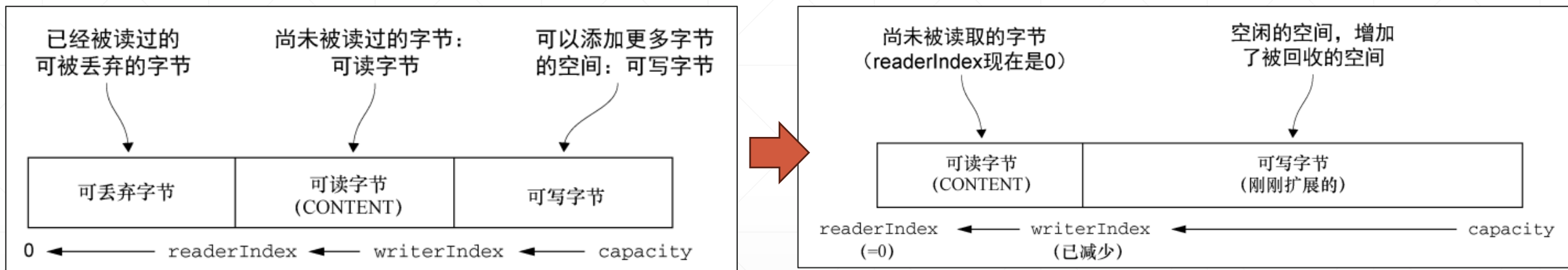


使用ByteBuf，关键就是要关注三个属性：readerIndex，writerIndex和capacity。

丢弃已读的数据



可以调用`discardReadBytes()`方法将已经读取的字节丢弃并回收其空间，相应地，读写指针前移：



如果你熟悉Java NIO中的ByteBuffer，不难发现这个操作，类似于ByteBuffer中的“compact”操作。

读写索引的示例 (ByteBufferIndex.java)

```
var byteBuf = Unpooled.buffer(10);  
//UnpooledByteBufferAllocator$InstrumentedUnpooledUnsafeHeapByteBuffer(ridx: 0, widx: 0, cap: 10)  
//初始值: readerIndex=writerIndex=0  
System.out.println(byteBuf);
```

写入数据之后，写索引变化：

```
//写入8个字节  
for (int i = 0; i < 8; i++) {  
    byteBuf.writeByte(i);  
}  
//UnpooledByteBufferAllocator$InstrumentedUnpooledUnsafeHeapByteBuffer(ridx: 0, widx: 8, cap: 10)  
//readerIndex不变, writerIndex=8  
System.out.println(byteBuf);
```

两种类型的读写操作



```
//使用get操作读取5个字节
for (int i = 0; i < 5; i++) {
    //get操作, 不会改变读索引
    System.out.println(byteBuf.getBytes(i));
}
//UnpooledByteBufAllocator$InstrumentedUnpooledUnsafeHeapByteBuf(ridx: 0, widx: 8, cap: 10)
//readerIndex=0
System.out.println(byteBuf);
```

```
//使用read操作读取5个字节
for (int i = 0; i < 5; i++) {
    //read操作, 会改变读索引
    System.out.println(byteBuf.readByte());
}
//UnpooledByteBufAllocator$InstrumentedUnpooledUnsafeHeapByteBuf(ridx: 5, widx: 8, cap: 10)
//readerIndex=5
System.out.println(byteBuf);
```



- read()和write()操作, 从给定的索引开始读写, 并且会根据已经访问过的字节数对索引值进行调整。
- get()和set()操作, 从给定的索引开始读写, 并且保持读写指针索引值不变。

ByteBuf与“容量”相关的API

`capacity()`

表示ByteBuf底层占用了多少字节的内存（包括丢弃的字节、可读字节、可写字节）

`maxCapacity()`

表示ByteBuf底层最大能够占用多少字节的内存，当向ByteBuf中写数据的时候，如果发现容量不足，则进行扩容，直到扩容到`maxCapacity`，超过这个数，就抛出异常。

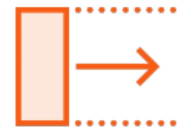
`readableBytes()`与`isReadable()`

`readableBytes()`表示ByteBuf当前可读的字节数，它的值等于`writerIndex-readerIndex`，如果两者相等，则不可读，`isReadable()`方法返回`false`。

`writableBytes()`、`isWritable()`与`maxWritableBytes()`

`writableBytes()`表示ByteBuf当前可写的字节数，它的值等于`capacity-writerIndex`，如果两者相等，则表示不可写，`isWritable()`返回`false`，但是这个时候，并不代表不能往ByteBuf写数据了。如果发现往ByteBuf写数据写不进去，Netty会自动扩容ByteBuf，直到底层的内存大小为`maxCapacity`，而`maxWritableBytes()`就表示可写的最大字节数，它的值等于`maxCapacity-writerIndex`。

ByteBuf的自动扩容特性



如果写入太多的数据.....

```
//接着上面已写入的8字节数据，再写入5个字节，很明显，这时，超过了原先设定的10个字节的容量
for (int i = 0; i < 5; i++) {
    byteBuf.writeByte(8+i);
}
//UnpooledByteBufAllocator$InstrumentedUnpooledUnsafeHeapByteBuf(ridx: 5, widx: 13, cap: 64)
//容量从原先的10，一下子跳到了64，读与写索引不变
System.out.println(byteBuf);
```

从上述代码可以看到，ByteBuf具有“自动扩容”特性，这点，大大地简化了开发，让开发者不需要依据特定的通讯协议去手动地调整ByteBuf的大小。



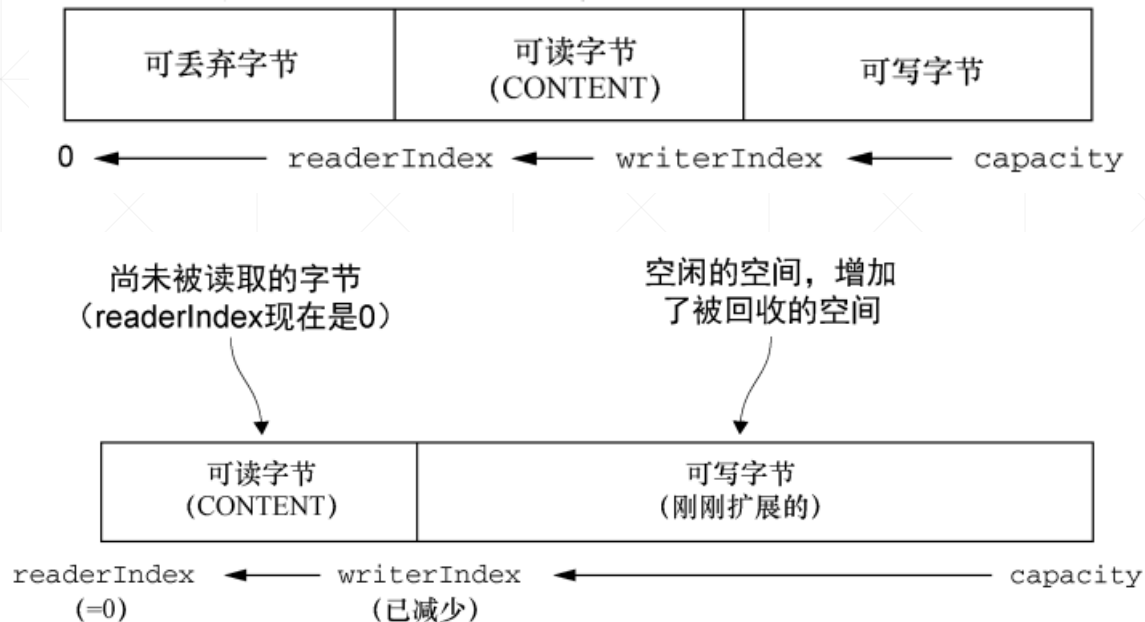
ByteBuf不能无限扩容，Netty定义有一个maxCapacity属性，限制了扩容的“上限”。



即读即弃示例

可以调用`discardReadBytes()`方法将已经读取的字节丢弃并回收其空间，相应地，读写指针前移。

```
//丢弃已经读过的数据
byteBuf.discardReadBytes();
//UnpooledByteBufAllocator$InstrumentedUnpooledUnsafeHeapByteBuf(ridx: 0, widx: 8, cap: 64)
//由于前5个已经被读过的数据被丢弃，现在，读写索引均有改变，值减少了5
//readerIndex原先为5，现在变成了0
//writerIndex原先为13，现在变成了8
System.out.println(byteBuf);
```



手动调整数据的读写索引值



可以通过调用`readerIndex(int)`或者`writerIndex(int)`来将索引移动到指定位置。



可以通过调用`markReaderIndex()`、`markWriterIndex()`、`resetWriterIndex()`和`resetReaderIndex()`来标记和重置`ByteBuffer`的 `readerIndex`和`writerIndex`，这个，实际上就是“书签”功能，例如：

```
// 代码片段一
int readerIndex = buffer.readerIndex();
// 其他操作
buffer.readerIndex(readerIndex);

// 代码片段二
buffer.markReaderIndex();
// 其他操作
buffer.resetReaderIndex();
```

左侧两段代码是等价的。



可以通过调用`clear()`方法来将`readerIndex`和`writerIndex`都设置为 0。

小结



本讲介绍了ByteBuf读写数据的基本方法，其实，思路是很直观的：

- ▶ ByteBuf定义有两个“读写指针”，随读写操作而改变。
- ▶ 要读或写特定位置的数据，先把指针移到那个位置，再进行读写。



下一讲，介绍ByteBuf中比较“高阶”的用法.....